

Amazon EKS Anywhere

Module 3 – Production Cluster Design

High Availability Architecture

Kubernetes Scheduling

Labels

Taints

Tolerations

Affinity

Anti-Affinity

Resource Requests

Resource Limits

Production Best Practices

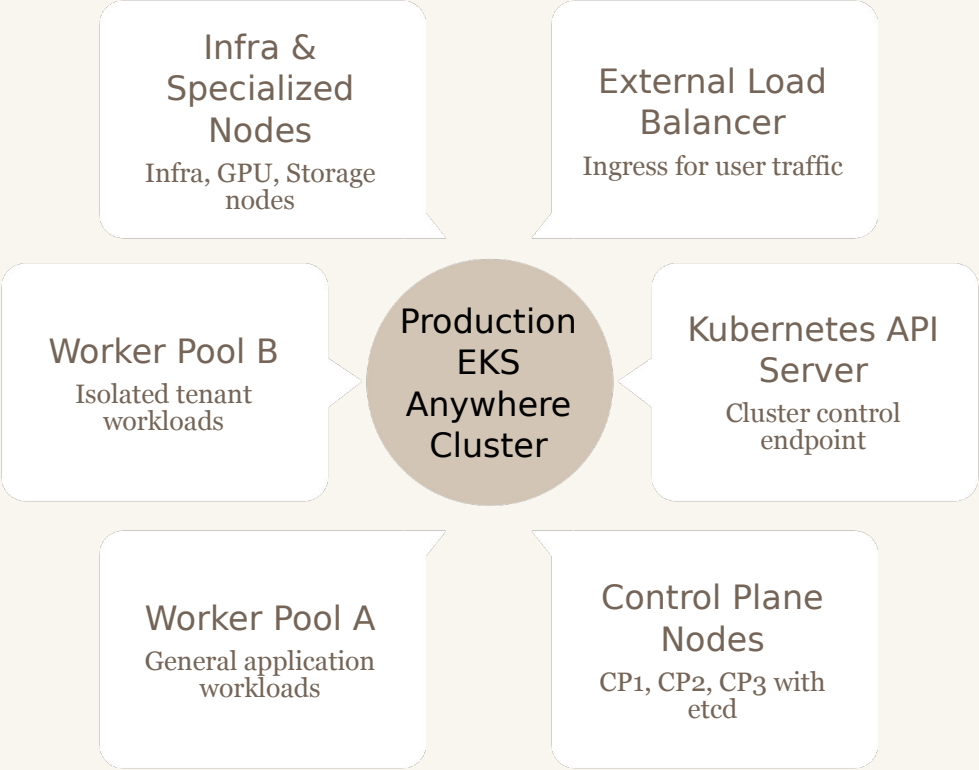
Learning Objectives

By the end of this module students will be able to

- ✓ Design a production-grade EKS Anywhere cluster
- ✓ Understand Kubernetes scheduling
- ✓ Separate workloads using labels and taints
- ✓ Control pod placement
- ✓ Allocate CPU and Memory correctly
- ✓ Build resilient applications
- ✓ Avoid common production mistakes

Production Cluster Architecture

Typical Production EKS Anywhere Cluster



Production clusters normally contain

Multiple Control Plane nodes	Multiple Worker Node Pools
Dedicated Infrastructure Nodes	Shared Storage
External Load Balancer	Monitoring Stack

High Availability Architecture

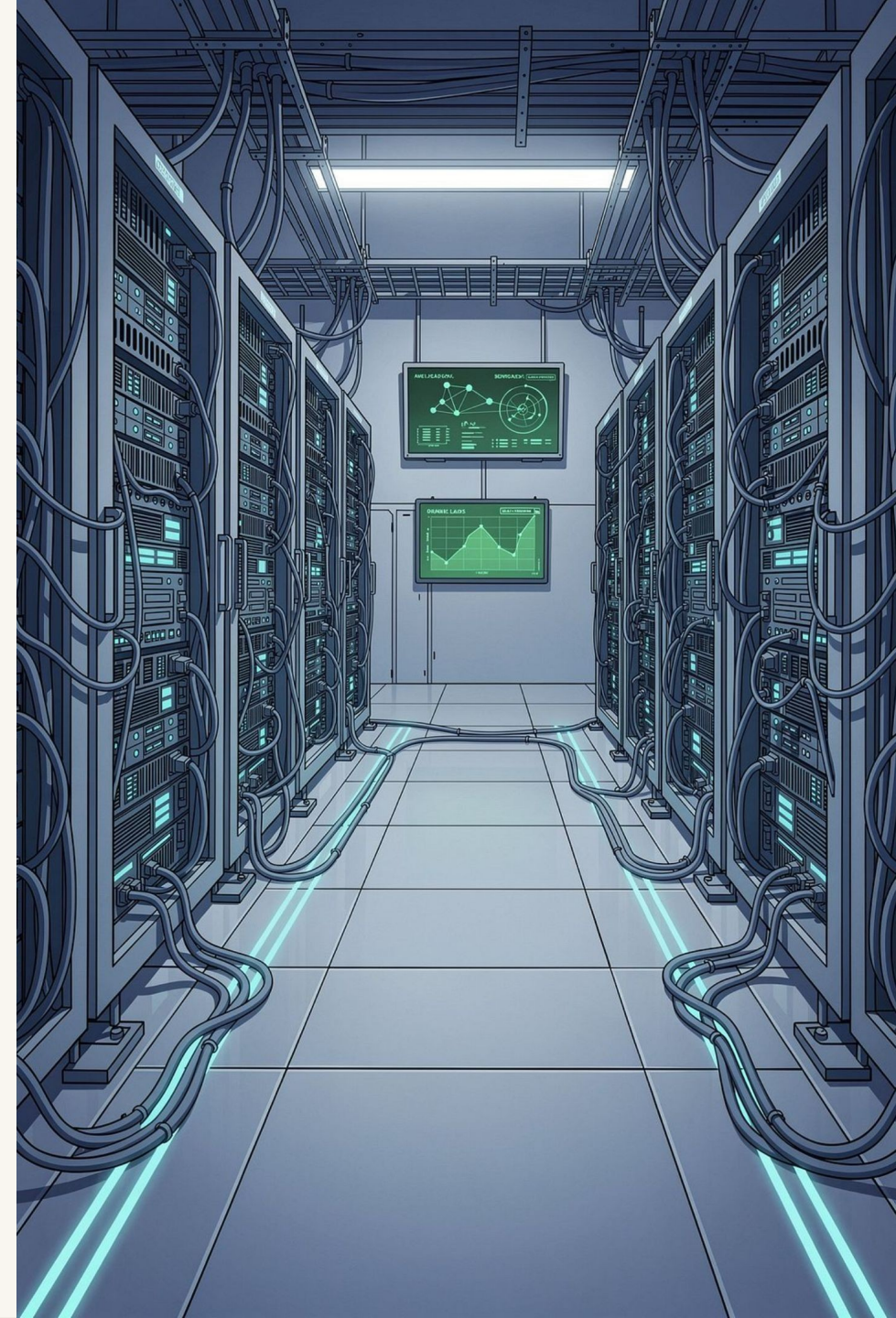
Why High Availability?

Production clusters must survive

- Node failure
- ESXi host failure
- VM reboot
- Kubernetes upgrade
- Hardware failure

Objectives

- Zero downtime
- Automatic failover
- Self-healing
- Rolling upgrades



HA Components

Component	HA Requirement
API Server	Multiple replicas
etcd	Odd number of members
Scheduler	Multiple
Controller Manager	Multiple
Worker Nodes	Multiple
Load Balancer	Required
Storage	Shared

 Never deploy a single Control Plane node in production.

Kubernetes Scheduler

The Kubernetes Scheduler decides

❏ Which worker node should run each Pod?

Scheduling happens every time a Pod is created.

Scheduler checks

Available CPU

Available Memory

Labels

Taints

Tolerations

Affinity Rules

Anti-Affinity Rules

Resource Requests

Resource Limits

Kubernetes Labels

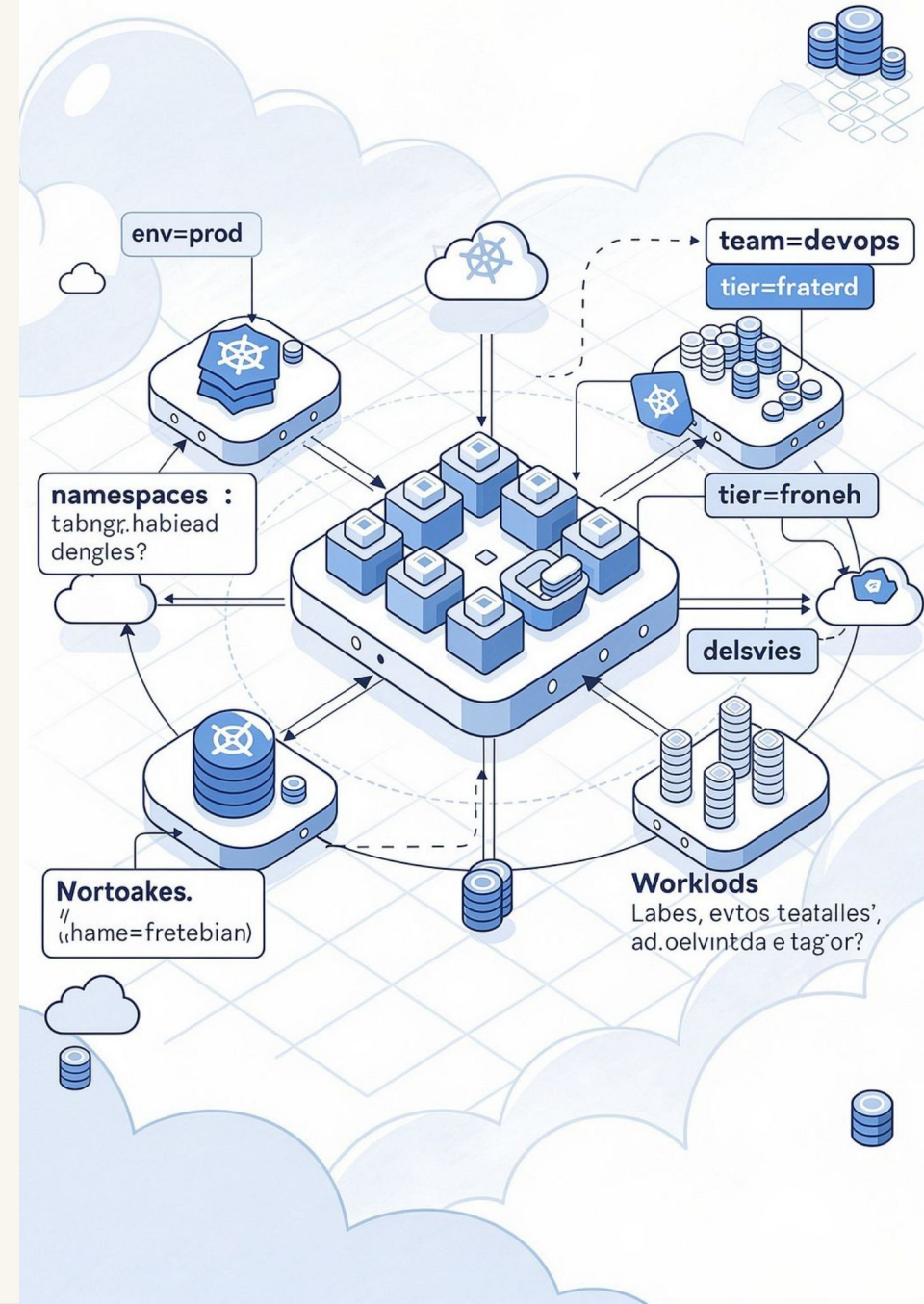
Labels are key-value pairs attached to Kubernetes objects.

Example

```
environment=production  
team=finance  
application=payment  
zone=az1  
role=worker
```

Labels help

- Identify resources
- Select resources
- Group workloads
- Target deployments



Label Examples

Worker Node

```
role=worker  
environment=production  
location=bangalore  
disk=ssd
```

GPU Node

```
role=gpu  
gpu=true  
vendor=nvidia
```

Database Node

```
role=database  
storage=fast
```

Applications can request specific labels.

Node Selectors

Simplest scheduling method

```
nodeSelector:  
  role: database
```

Scheduler only selects nodes matching

```
role=database
```

Advantages

Simple

Fast

Easy to understand

Limitations

- Cannot express preferences
- Cannot rank nodes

Taints

Taints repel Pods.

📄 Think of a taint as "Do NOT schedule Pods here."

Example

```
kubectl taint nodes worker03 \
dedicated=database:NoSchedule
```

Result

Only approved Pods can run on this node.

Useful for

- Databases
- GPU
- Storage
- Infrastructure services



Tolerations

Tolerations allow Pods to ignore taints.

Example

```
tolerations:  
- key: dedicated  
  value: database  
  operator: Equal  
  effect: NoSchedule
```

Now the Pod can run on

✔ dedicated=database

Labels + Taints Together

Production Example

GPU Nodes

```
label: gpu=true  
taint: gpu=true:NoSchedule
```

AI workload

```
nodeSelector  
  gpu=true  
  
plus  
  
toleration  
  gpu=true
```

Result

Only AI Pods use GPU servers.

Affinity

Affinity attracts Pods toward specific nodes.

Example

Run frontend Pods only on SSD nodes

Example

```
nodeAffinity
  requiredDuringScheduling
```

Benefits

Flexible

Powerful

Preferred scheduling

Supports multiple
conditions

Anti-Affinity

Anti-affinity keeps Pods apart.

Example

Three replicas

Web-1 Web-2 Web-3

Scheduler places them on

Worker01 Worker02 Worker03

instead of

Worker01 Worker01 Worker01

Benefits

High Availability

Fault Tolerance

Load Distribution

Resource Requests

Requests tell Kubernetes

- ☐ Minimum resources required.

Example

```
resources:  
  requests:  
    cpu: "500m"  
    memory: "512Mi"
```

Scheduler guarantees

- CPU
- Memory

before scheduling.

⚠ Without requests — Pods may compete for resources.

Resource Limits

Limits define

- ❏ Maximum resource consumption.

Example

```
resources:  
  limits:  
    cpu: "2"  
    memory: "2Gi"
```

Benefits

Prevent noisy
neighbors

Prevent memory
exhaustion

Avoid CPU starvation

Protect cluster
stability

Production Best Practices

Labels

✓ Consistent naming

✓ Team labels

✓ Environment labels



Production Best Practices

Taints

✓ Reserve Infra Nodes

✓ Reserve GPU Nodes

✓ Separate Monitoring

Production Best Practices

Affinity

✓ Spread workloads

✓ Keep applications close

Production Best Practices

Resources

✓ Always define Requests

✓ Always define Limits